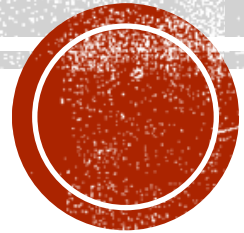
Faculty of Economics and Business Administration

Introduction to Mobile Applications using flutter



Dr. Abdul Rahman EL SAYED

OBJECTIVES

- This course is meant as an introduction to creating Mobile applications.
- We will learn how to create a mobile GUI application and deploy them to emulators and mobile devices.
- Students will learn how to connect their applications to a remote database through RESTFUL services.
- Students will learn the above concepts through the Flutter framework.

TOPICS

- Dart programming language basics.
- Creating Mobile GUI applications using Flutter.
- Creating online databases.
- How to create and access REST services.
- Publishing your projects using a source control system (GIT).

WHAT IS DART?

- Flutter uses the Dart programming language.
- Dart was original designed to create web applications.
- Originally Dart was compiled to JavaScript.
- Now Dart can be compiled to many platforms, such as Android, IOS, Windows and Linux.

HOW TO USE DART?

- You can start writing and testing Dart code using online service (dartpad.dev).
- You can download the Flutter SDK, which includes the Dart SDK from flutter.dev.
- You can use a cloud based platform, such as replit.com.
- Optionally, you may download Visual studio code (code.visualstudio.com). It is useful for writing Dart code locally on your computer.

WHAT IS FLUTTER?

- Flutter is a framework to deploy applications from a single source code to a variety of platforms.
- You can deploy Flutter applications to:
  - Android
  - IOS
  - Windows
  - Linux
  - Web
- You can use a variety of IDE's to write your applications, such as:
  - Android Studio
  - VS Code

ADVANTAGES OF FLUTTER

- Can be deployed to multiple platforms.
- Compiles to native machine code, so it is fast and efficient.
- Has a hot-reload feature, which increases the speed of testing and deploying your applications.
- Fast rendering of GIU components. It allows partial refresh of screen components, which makes screen rendering faster.

INSTALLING AND CONFIGURING FLUTTER

- The minimum requirements is 8GB of RAM and Windows 10 or above.
- You need to install the Flutter SDK, which includes the Dart SDK.
- You need to install Git, since Flutter relies heavily on it.
- You need to configure the Flutter SDK (https://docs.flutter.dev/get-started/install/windows).
- Since will be developing applications for Android, you need to install Android Studio and Android Emulator. (https://developer.android.com/studio).
- Note: You do not need the android SDK, since we will not be writing android applications using Java.

SOURCE VERSION CONTROL PLATFORMS

- Before the age of Source Version Control platforms, a programmer would do the following while working on a project:

1. Create a directory for the project
2. Write some working code so that the project implements some features
3. Backup the project before adding more features.

The above tasks become harder when we are working as a group with multiple other programmers.

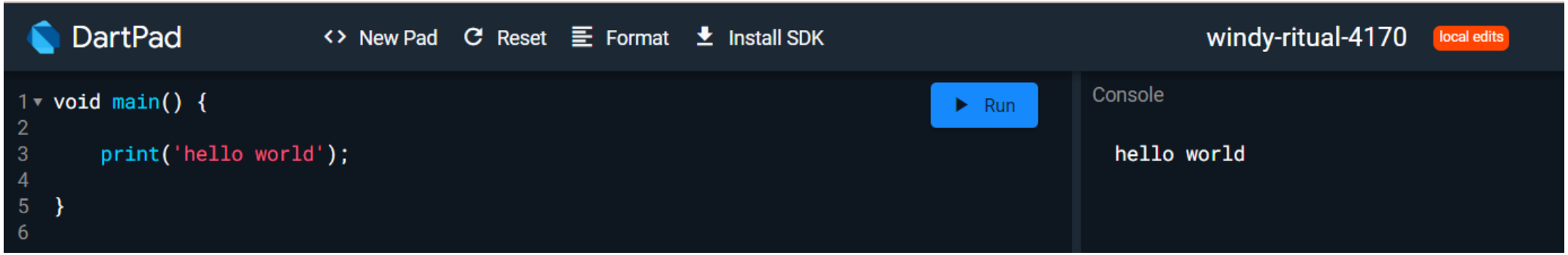
A source version control platform is designed to automate the above tasks and to allow multiple programmers to work on the same code base.

GIT AND GITHUB

- Git is one of the mostly used source version control platforms.
- Git and GitHub are not the same.
- Git contains the tools for version control, while GitHub is an online cloud system that allows programs to store their source code. You can use Git with other hosting platforms such as:
  - BitBucket
  - GitLab
- You projects must be published on GITHUB.
- You can learn Git on https://www.w3schools.com/git
- You can download Git from https://git-scm.com/downloads

WRITING YOUR FIRST DART PROGRAM

- You can go to dartpad.dev and write your first hello world program.



Dart is both procedural and an OOP language. This means that declaring classes is optional.

You should have at least one function in your program called main. It's the starting point of your program.

Dartpad translates Dart code to JavaScript to be run inside a web browser.

Dartpad can't take user console input. You need to install the Dart SDK, to take user console input.

You can also use replit that contains a pre-installed Dart SDK running on Linux.

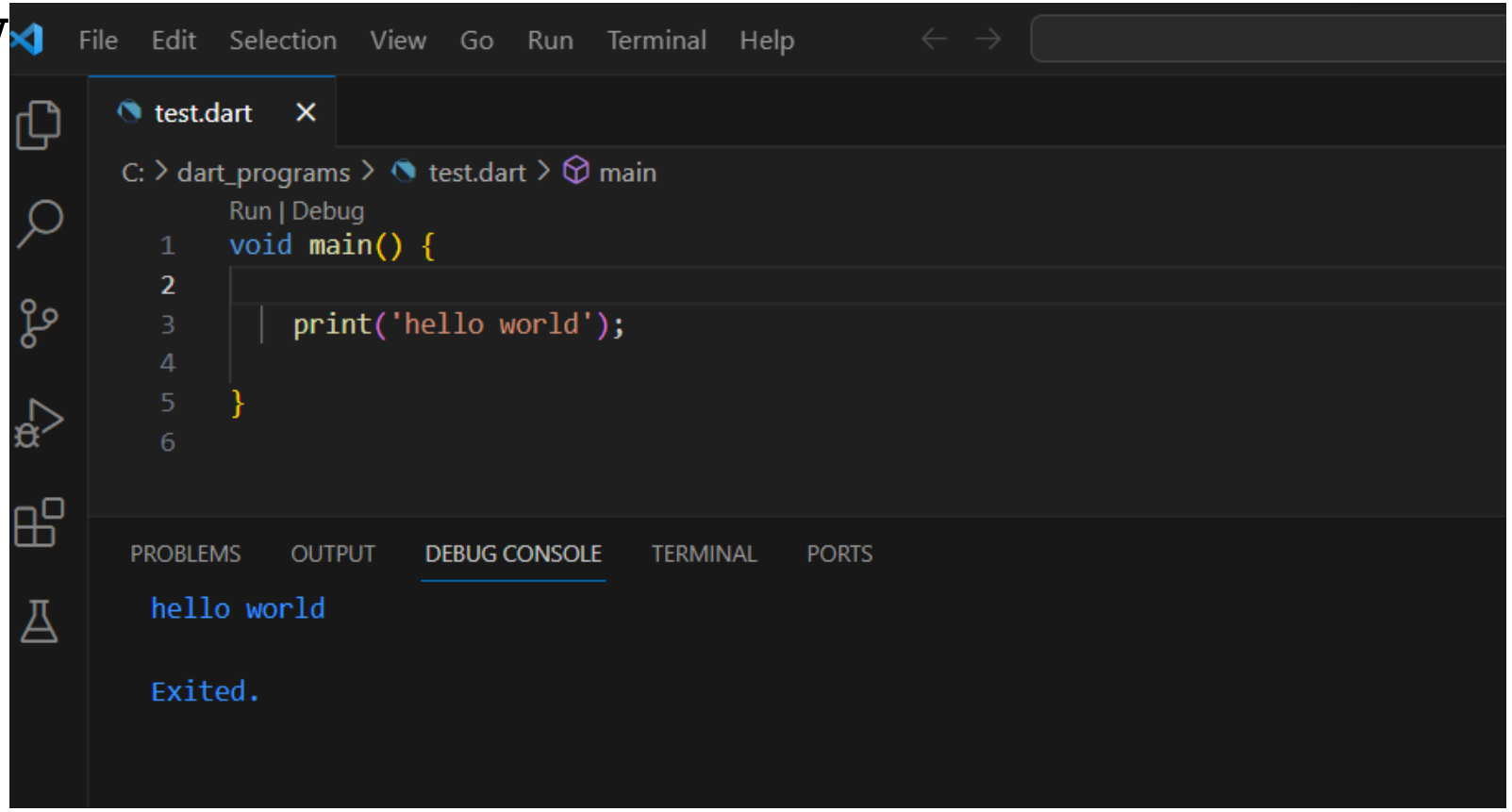
RUNNING DART PROGRAMS USING SDK

- You can write Dart programs using your favorite text editor such as notepad. You can then save your file and execute it using dart command.

The above assumes you saved your code inside a file called test.dart inside the dart_programs folder.

RUNNING DART PROGRAMS USING VS CODE

- You can write Dart programs using Visual Studio Code with the Dart plugin, as show



COMPILE YOUR PROGRAM TO MACHINE CODE

- You can compile your code to an executable .exe file on windows. Then you can run this executable program, without using the dart command again.
- This is faster than using the dart command.
- You need to re-compile your program each time you make changes to the source code.

ADDING 2 NUMBERS

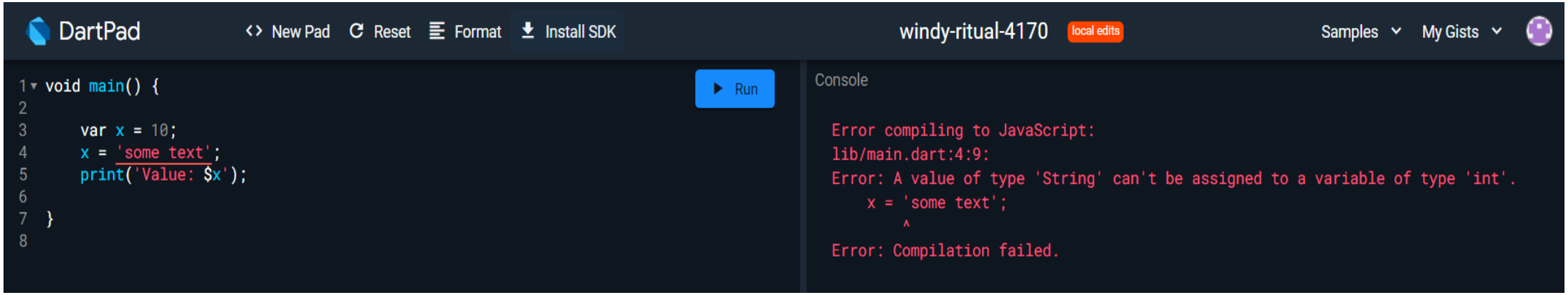
```
void main() {
   var x = 10;
   var y = 20;
   var sum = x + y;
   // print adds a new line after the
   // printed string
   print('Result: $sum');
}
```

```
void main() {
   int x = 10;
   int y = 20;
   int sum = x + y;
   // $ sign forces dart to use the variable value and
converts
   // it to a string
   print('Result: $sum');
}
```

- Dart is a strongly typed language, although its optional to declare variable types. Dart automatically infers a variable type from its assigned value. In the above case it will set x & y to integer types.
- You can optionally define a variable type for clarity, as shown in the above right side code.

DART IS A STRONGLY TYPED LANGUAGE

- Dart is a strongly typed language. This means once the variable type is inferred, you can't assign values of different types to it. In the below code, because we assigned an integer to the variable x, Dart will automatically define its type as integer. If you try to assign a string value afterwards, this will generate a compile time error.



DART ARITHMETIC OPERATORS

Dart comes with many typical operators that work like many languages; this includes the following:

- +: This is for the addition of numbers. It can also be used to concatenate strings.
- -: This is for subtraction.
- *: This is for multiplication.
- /: This is for division.
- ~/: This is for integer division. In Dart, any simple division with / results in a double value. To get only the integer part, you would need to make some kind of transformation (that is, type cast) in other programming languages; however, here, the integer division operator does this task.
- %: This is for modulo operations (the remainder of integer division).
- -expression: This is for negation (which reverses the sign of expression).

EQUALITY AND RELATIONAL OPERATORS

The equality Dart operators are as follows:

- ==: For checking whether operands are equal
- !=: For checking whether operands are different

For relational tests, the operators are as follows:

- >: For checking whether the left operand is greater than the right one
- =: For checking whether the left operand is greater than or equal to the right one
- <=: For checking whether the left operand is less than or equal to the right one

Note: In Dart, unlike Java and many other languages, the == operator does not compare memory references but rather the content of the variable.

DART DATA TYPES

- final and const: The value variable cannot be changed once it's initialized.
- int: 64-bit signed non-fractional integer values such as -2^{63} to $2^{63}-1$.
- double: Dart represents fractional numeric values with a 64-bit double-precision floating-point number.
- Bool: Its value can either be true or false.
- BigInt: Dart also has the BigInt type for representing arbitrary precision integers, which means that the size limit is the running computer's RAM. This type can be very useful depending on the context; however, it does not have the same performance as num types and you should consider this when deciding to use it.
- Strings: In Dart, strings are a sequence of characters (UTF-16 code) that are mainly used to represent text

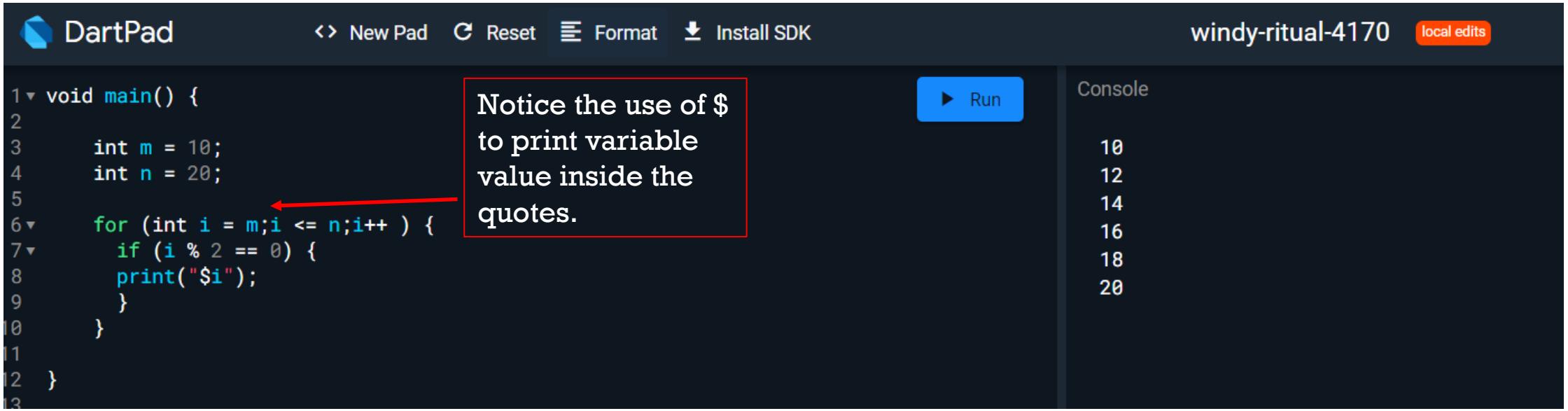
CONTROL FLOWS AND LOOPING

We've reviewed how to use Dart variables and operators to create conditional expressions. To work with variables and operators, we typically need to implement some control flow to make our Dart code take the appropriate direction in our logic. Dart provides some control flow syntax that is very similar to other programming languages; it is as follows:

- if-else
- switch/case
- Looping with for, while, and do-while
- break and continue

The above are similar in use to Java, so we will not go over them in details.

EXAMPLE: PRINTING THE EVEN NUMBERS BETWEEN M & N.




```
void main() {

    int m = 10;
    int n = 20;

    for (int i = m;i <= n;i++ ) {
      if (i % 2 == 0) {
      print("$i");
      }
    }

}
```

CONSOLE INPUT

- Dartpad compiles source code to JavaScript. We can't take console input using dartpad. We need to use the Dart SDK. Below is an example written in visual studio code.
- Dart is null safe, so when we declare the String values we have to add the "?" Symbol to the end of the variable type., since in the below code, first and last variables may have a null value.

l2.dart > main

```dart
import 'dart:io';

// The above package is required for write and readLIneSync functions
Run | Debug
void main() {
  String? first, last; // we need to use ? becase its value may be null
  stdout.write('Enter first name: '); // stdout.write does not create a new
  // kine after the string
  first = stdin.readLineSync(); // this will wait for user input
  stdout.write('Enter last name: ');
  last = stdin.readLineSync();
  print('Welcome $first $last');
}

```

TAKING INTEGER INPUT

- When you use readLineSync to take integer input, you need to convert it to an integer using int.parse function.
- Since readLIneSync may return null, we need to use the null assertion operator (!) to make Dart treat it as non-nullable.

l2.dart > main

```dart
import 'dart:io';

// The above package is required for write and readLIneSync functions
Run | Debug
void main() {
  int x, y;
  stdout.write('Enter x: ');
  x = int.parse(stdin.readLineSync()!);
  stdout.write('Enter y: ');
  y = int.parse(stdin.readLineSync()!);
  print('Sum is ${x + y}');
}

```